Indian Institute of Technology Dharwad

Pattern Recognition and Machine Learning (EE602L)

Lab 4

Multiclass Logistic Regression and Support Vector Machines

Roll Number: IS24BM008

Submitted files

`Lab4-IS24BM008.ipynb`	commented Google Colab notebook
`Lab4-IS24BM008.pdf`	this report

Contents

1 Overview and experimental setup **2**

2 Question 1 — Multiclass logistic regression via softmax **2**

2.1 Model, cost function and gradient 2

2.2 (a) Data preparation 3

2.3 (b) Training and learning-rate selection 4

2.4 (b) Verifying the reduction to the sigmoid at $K = 2$ 6

2.5 (c) Evaluation on the test set 6

2.6 (c) Comparison with `sklearn.linear_model.LogisticRegression` 7

3 Question 2 — Support vector machines **8**

3.1 Margins, support vectors and the role of C 8

3.2 (a) Geometric intuition on synthetic 2D data 9

3.3 (b) Effect of regularisation on classification 10

3.4 (c) Full evaluation on the Breast Cancer Wisconsin dataset 12

4 Conclusions **15**

1 Overview and experimental setup

This report documents the two experiments of Lab 4.

Question 1 implements multiclass logistic regression through the softmax function *entirely from scratch* in NumPy — the stratified data split, the standardisation, the one-hot encoding, the softmax, the cross-entropy cost, its analytic gradient, batch gradient descent, and every evaluation metric (confusion matrix, precision, recall, F1 and multiclass log-loss). The model is trained on the Wine dataset, the learning rate is selected on a held-out validation set, the reduction of the softmax to the binary sigmoid at $K = 2$ is verified numerically, and the final model is compared against `sklearn.linear_model.LogisticRegression`.

Question 2 studies linear support vector machines: the geometry of the margin and of the support vectors on synthetic two-dimensional data, the effect of the regularisation parameter C on the width of the margin and on the classification error, and a full evaluation of a linear SVM on the Breast Cancer Wisconsin dataset.

Every experiment uses the fixed random seed 42. scikit-learn is used only to load the datasets and to supply the reference implementations we are explicitly asked to compare against (`LogisticRegression` in Q1(c), `SVC` in the whole of Q2); all the Question 1 machinery is our own. All numbers and all figures in this report are the direct output of the accompanying notebook `Lab4-IS24BM008.ipynb`, executed top to bottom in one pass.

2 Question 1 — Multiclass logistic regression via softmax

2.1 Model, cost function and gradient

Let $x \in \mathbb{R}^{n+1}$ be a feature vector with a leading 1 for the intercept, and let $\Theta \in \mathbb{R}^{(n+1)\times K}$ collect one parameter column per class. The scores (logits) are $z = \Theta^\top x$ and the softmax turns them into a probability distribution over the K classes:

$$P(y = k \mid x; \Theta) \;=\; \frac{\exp(z_k)}{\sum_{j=1}^{K} \exp(z_j)}, \qquad k = 1, \dots, K. \tag{1}$$

With one-hot targets $Y \in \{0, 1\}^{m\times K}$ the average multiclass cross-entropy cost is

$$J(\Theta) \;=\; -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{K} Y_{ik} \log P_{ik}, \qquad P_{ik} = P(y = k \mid x^{(i)}; \Theta). \tag{2}$$

Differentiating (2) through (1) gives the remarkably compact gradient

$$\nabla_\Theta J \;=\; \frac{1}{m} X^\top (P - Y) \;\in\; \mathbb{R}^{(n+1)\times K}, \tag{3}$$

and batch gradient descent is the single update $\Theta \leftarrow \Theta - \alpha\, \nabla_\Theta J$, where every iteration uses the whole training set.

Two numerical points are handled explicitly in the implementation. The softmax is evaluated after subtracting the row maximum from the logits, which leaves (1) unchanged but prevents exp from overflowing; and the cost is evaluated in the log-sum-exp form $J = \frac{1}{m} \sum_i \left[\log \sum_k e^{z_{ik}} - z_{i,y_i} \right]$, which never takes the logarithm of a number that has underflowed to zero.

```
def softmax(Z):
    Z_shift = Z - Z.max(axis=1, keepdims=True)   # stability shift
    expZ = np.exp(Z_shift)
```

```
    return expZ / expZ.sum(axis=1, keepdims=True)

def cross_entropy_gradient(X, Y, theta):
    m = X.shape[0]
    P = softmax(X @ theta)
    return (X.T @ (P - Y)) / m                    # (1/m) X^T (P - Y)
```

2.2 (a) Data preparation

The Wine dataset contains 178 samples, 13 numerical features and 3 classes, with no missing values. Table 1 shows the class balance: the classes are moderately but not severely imbalanced, class 1 being about 1.5 times as frequent as class 2. Because no class is rare, macro averaging of precision, recall and F1 is meaningful and accuracy is not a misleading summary.

Table 1: Class balance of the full Wine dataset.

Class	Count	Proportion
class_0	59	0.3315
class_1	71	0.3989
class_2	48	0.2697
Total	178	1.0000

Table 2 lists the summary statistics of the raw features. The scales are wildly different — `proline` has a mean of about 747 and a standard deviation of about 315, while `nonflavanoid_phenols` has a mean of 0.362. Gradient descent on such raw features would be dominated by `proline` and would converge extremely slowly along the other directions, which is precisely why standardisation is required.

Table 2: Summary statistics of the 13 raw Wine features.

Feature	Mean	Std	Min	Median	Max
`alcohol`	13.001	0.812	11.03	13.050	14.83
`malic_acid`	2.336	1.117	0.74	1.865	5.80
`ash`	2.367	0.274	1.36	2.360	3.23
`alcalinity_of_ash`	19.495	3.340	10.60	19.500	30.00
`magnesium`	99.742	14.282	70.00	98.000	162.00
`total_phenols`	2.295	0.626	0.98	2.355	3.88
`flavanoids`	2.029	0.999	0.34	2.135	5.08
`nonflavanoid_phenols`	0.362	0.124	0.13	0.340	0.66
`proanthocyanins`	1.591	0.572	0.41	1.555	3.58
`color_intensity`	5.058	2.318	1.28	4.690	13.00
`hue`	0.957	0.229	0.48	0.965	1.71
`od280/od315_of_diluted_wines`	2.612	0.710	1.27	2.780	4.00
`proline`	746.893	314.907	278.00	673.500	1680.00

Split. The split is written from scratch. For each class separately the indices are shuffled with a seeded generator and then cut at 70% and 85%, so the class proportions are preserved in every subset; the three index blocks are finally reshuffled so that the subsets are not ordered by class. The resulting sizes are 125 / 27 / 26 (Table 3), i.e. 70.2%/15.2%/14.6% — the small

deviation from exactly 70:15:15 is unavoidable because the per-class counts 59, 71 and 48 are not divisible in that ratio. The notebook asserts that the three index sets are pairwise disjoint and together cover all 178 samples.

Table 3: Stratified 70:15:15 split of the Wine dataset (seed 42).

Subset	n	class_0	class_1	class_2	Fraction
Train	125	41	50	34	0.7022
Validation	27	9	11	7	0.1517
Test	26	9	10	7	0.1461
Total	178	59	71	48	1.0000

Standardisation and encoding. The mean and standard deviation are computed on the *training subset only* and then applied unchanged to the validation and test subsets. After scaling, the training features have mean 0 and standard deviation 1 to machine precision, while the validation features have means such as 0.048 and -0.218 — they are deliberately *not* exactly zero, which is the visible sign that no information from the held-out data has leaked into the scaler. The targets are one-hot encoded into a 125×3 matrix for training, while the integer labels are retained for evaluation.

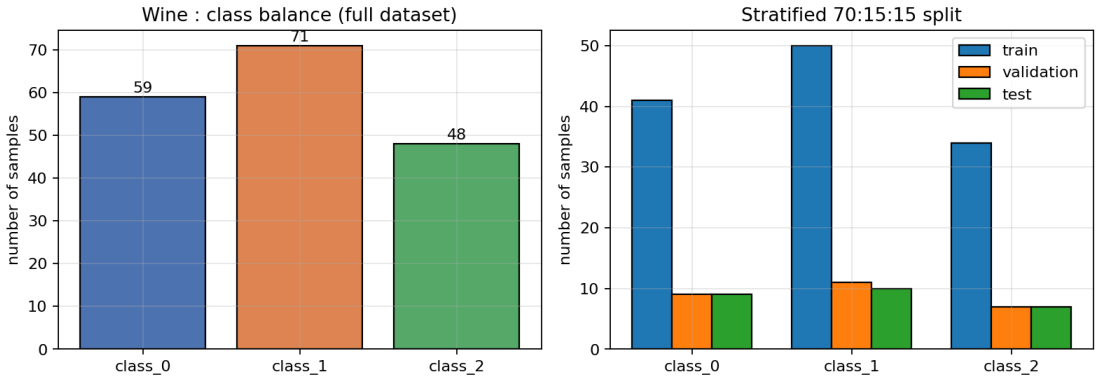


Figure 1: Left: class balance of the full Wine dataset. Right: the same classes after the stratified 70:15:15 split; the proportions are preserved in each subset.

2.3 (b) Training and learning-rate selection

Correctness of the gradient. Before any training, the analytic gradient (3) is checked against central finite differences $\big(J(\theta+\varepsilon) - J(\theta-\varepsilon)\big)/2\varepsilon$ with $\varepsilon = 10^{-6}$ on a small random problem. The relative error is 5.53×10^{-10}, which is at the level expected from floating-point truncation alone and confirms that the implemented gradient is exact.

Learning-rate sweep. The model is trained with four learning rates spanning nearly three orders of magnitude, $\alpha \in \{0.005, 0.05, 0.5, 1.5\}$, each for 3000 full-batch iterations from a zero initialisation (the cost is convex, so the starting point does not affect the optimum). The results are in Table 4 and Figure 2.

Table 4: Learning-rate sweep, 3000 batch gradient descent iterations each. The selected row (lowest validation cross-entropy) is in bold.

α	Final train CE	Final val. CE	Train acc.	Val. acc.
0.005	0.078832	0.076045	0.9920	1.0000
0.05	0.016519	**0.051590**	1.0000	0.9630
0.5	0.002421	0.069817	1.0000	0.9630
1.5	0.000871	0.086955	1.0000	0.9630

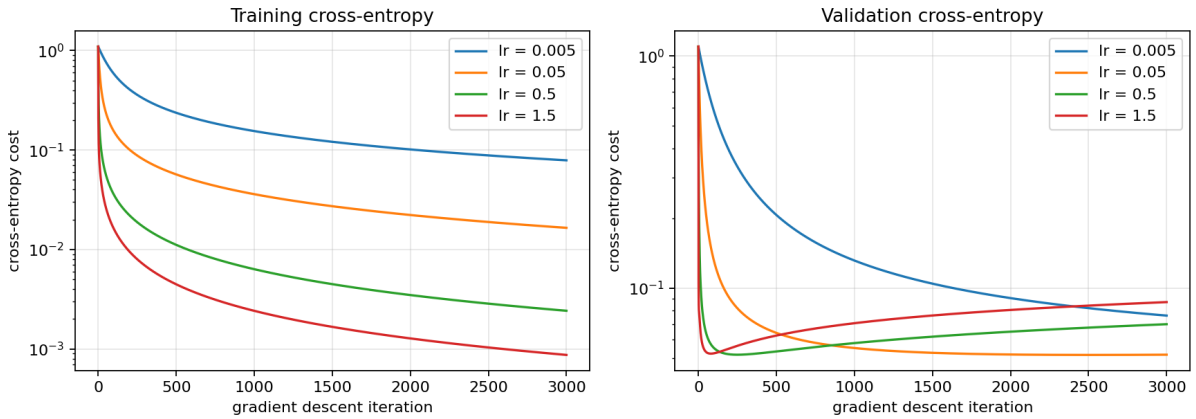


Figure 2: Cross-entropy against iteration for the four learning rates (log vertical axis). Training cost decreases monotonically for every rate, but the validation cost of the two largest rates reaches a minimum and then rises again.

Figure 2 is the most informative plot of Question 1. On the left the training cost falls monotonically for all four rates, and the larger the rate the lower it gets — $\alpha = 1.5$ reaches a training cross-entropy of 0.000871, essentially interpolating the training set. None of the rates diverges, so all four are within the stable region $\alpha < 2/L$ for this problem.

On the right the picture is completely different. The validation cost of $\alpha = 0.5$ and $\alpha = 1.5$ falls to a minimum after a few hundred iterations and then *increases* again, ending at 0.086955 for $\alpha = 1.5$. The four minima themselves are almost identical — 0.0515, 0.0515 and 0.0521 for $\alpha = 0.05$, 0.5 and 1.5, reached at iterations 2516, 250 and 80 respectively — but the amount by which each curve then climbs back is not: 1.2×10^{-4} for $\alpha = 0.05$ against 0.018 for $\alpha = 0.5$ and 0.035 for $\alpha = 1.5$ ($\alpha = 0.005$ is still descending at iteration 3000). In other words, all four trajectories pass through essentially the same best model; the learning rate only decides how quickly they pass it and how far past it they travel in the remaining budget. This is overfitting made visible: because the Wine classes are linearly separable in the standardised 13-dimensional feature space and the cost carries no regularisation term, the likelihood can always be increased by scaling the weights up, which drives the predicted probabilities towards 0 and 1 and is punished severely on any validation point that falls on the wrong side. With a fixed iteration budget, the learning rate therefore acts as an implicit regulariser — a larger step size travels further along that path.

Selecting on validation cross-entropy, as instructed, gives $\boxed{\alpha = 0.05}$ with a validation cost of 0.051590. It is worth noting that $\alpha = 0.005$ attains a *higher* validation accuracy (1.0000 against 0.9630) while having a worse validation cross-entropy. The two criteria genuinely disagree: cross-entropy is a proper scoring rule that also grades confidence, whereas accuracy only grades the position of the argmax, and on 27 validation samples one sample is worth 3.7 percentage points of accuracy. The instruction is to select on validation cross-entropy, which is the more

informative of the two on a set this small, so $\alpha = 0.05$ is carried forward.

2.4 (b) Verifying the reduction to the sigmoid at $K = 2$

The softmax is over-parameterised: adding the same vector to every column of Θ leaves (1) unchanged, so only the *difference* of the columns is identifiable. This is exactly what allows the reference column to be fixed at $\theta_1 = 0$ (in the notation of the problem sheet, $\theta_2 = 0$ after relabelling). Doing so gives, for $K = 2$,

$$P(y = 2 \mid x) = \frac{e^{z_2}}{e^{z_1} + e^{z_2}} = \frac{1}{1 + e^{-(z_2 - z_1)}} = \sigma(z_2 - z_1), \tag{4}$$

which is the binary logistic model with weight vector $\theta_2 - \theta_1$. The notebook verifies this in two independent ways.

1. **The identity itself.** On 2000 random score pairs drawn with a standard deviation of 6 (so the probabilities span the full range from ≈ 0 to ≈ 1), the largest absolute difference between $\mathrm{softmax}(z)_2$ and $\sigma(z_2 - z_1)$ is exactly 0 — the two expressions agree bit for bit in double precision.

2. **The trained models.** A two-class subset of Wine (classes 0 and 1, 130 samples, 91 of them in the training part) is used to train two models with identical learning rate (0.5), identical iteration count (800) and identical zero initialisation:

   - softmax regression with $K = 2$ in which the gradient column of the reference class is zeroed at every step, so that θ_1 stays at 0;
   - ordinary binary logistic regression driven by the sigmoid, with gradient $\frac{1}{m} X^\top (\sigma(Xw) - y)$.

   The largest absolute difference between the free softmax column and the binary weight vector is 2.22×10^{-16}, and the largest difference between the two sets of predicted probabilities is 4.16×10^{-17}. Both are at the level of a single floating-point rounding unit, i.e. the two procedures are not merely similar but algebraically identical.

2.5 (c) Evaluation on the test set

Each test sample is assigned the class $\arg\max_k P(y = k \mid x)$. On the 26 held-out samples the from-scratch model produces the confusion matrix

	pred class_0	pred class_1	pred class_2
true class_0	9	0	0
true class_1	0	10	0
true class_2	0	0	7

which is perfectly diagonal (Figure 3). Every per-class precision, recall and F1 is therefore 1, and so is every macro average (Table 5). The manually computed multiclass log-loss is 0.007035.

Table 5: Per-class test metrics of the from-scratch softmax model ($\alpha = 0.05$).

Class	Support	Precision	Recall	F1
class_0	9	1.0000	1.0000	1.0000
class_1	10	1.0000	1.0000	1.0000
class_2	7	1.0000	1.0000	1.0000
Macro average	26	1.0000	1.0000	1.0000

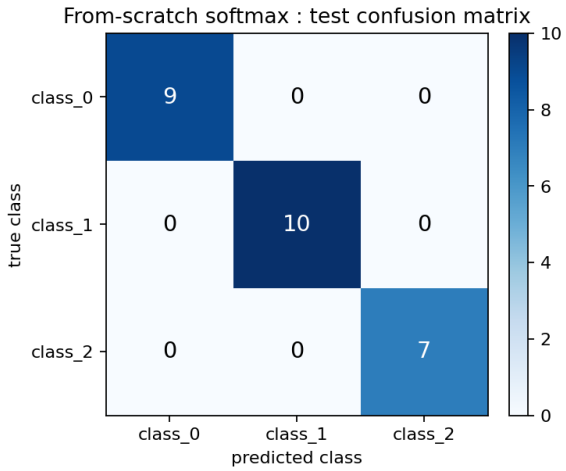


Figure 3: Test confusion matrix of the from-scratch softmax model. All 26 test samples are classified correctly.

A perfect test score is a claim that needs qualifying rather than celebrating. The test set holds only 26 samples, so the 95% Wilson interval around an observed accuracy of 1.0 still extends down to roughly 0.87; the honest reading is "no errors were observed on 26 samples", not "the error rate is zero". Wine is a small, clean, low-noise dataset whose three cultivars are close to linearly separable once the features are standardised, so a linear model doing this well is expected.

2.6 (c) Comparison with `sklearn.linear_model.LogisticRegression`

The library model is fitted on exactly the same standardised training matrix with its default settings: the multinomial (softmax) formulation, an L2 penalty with $C = 1$, and the quasi-Newton `lbfgs` solver, which converged in 17 iterations. Table 6 compares the two on the test set.

Table 6: From-scratch softmax regression against `sklearn.linear_model.LogisticRegression` on the Wine test set.

Metric	From-scratch softmax	`sklearn`
Accuracy	1.0000	1.0000
Macro precision	1.0000	1.0000
Macro recall	1.0000	1.0000
Macro F1	1.0000	1.0000
Log-loss	0.007035	0.022759
$\|W\|_F$ (excluding bias)	4.875	3.338

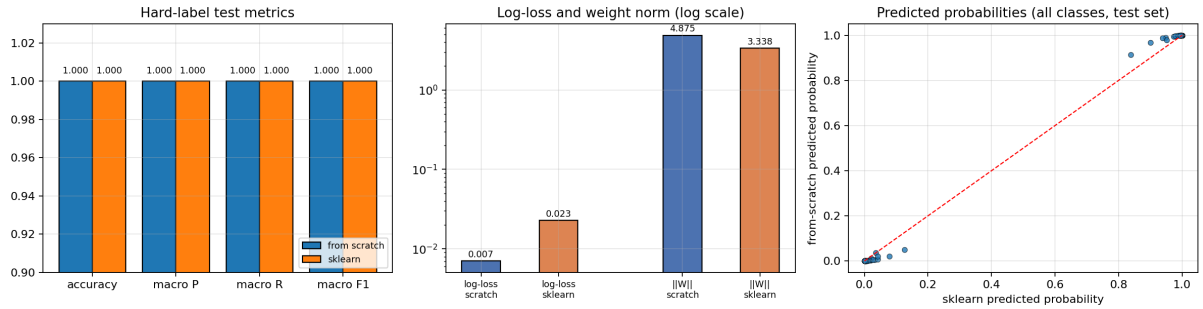


Figure 4: Left: the hard-label test metrics are identical. Centre: log-loss and weight norm, where the two models genuinely differ. Right: the predicted probabilities of the two models, plotted against each other with the identity line; the from-scratch points sit slightly above the line in the upper corner and below it in the lower corner, i.e. they are more extreme.

Discussion of the differences. The two models agree on *every single test prediction* (agreement 1.0000), so all four hard-label metrics coincide at 1.0000. Where they differ is in confidence:

- **Log-loss.** Ours is 0.007035, the library's is 0.022759 — more than three times larger. The largest difference between two predicted probabilities is 0.0766. Since both models are perfectly accurate, a lower log-loss simply means our model commits harder to the correct class.
- **Weight norm.** $\|W\|_F = 4.875$ for our model against 3.338 for the library's. This is the cause of everything above. The library minimises $\frac{1}{2}\|W\|^2 + C\sum_i \ell_i$ with $C = 1$, so it pays for large weights; our cost (2) is unregularised, so on separable data nothing stops the weights from growing and the probabilities from saturating.
- **Optimiser.** `lbfgs` is a second-order method that uses curvature information and terminates on a gradient tolerance (here after only 17 iterations); ours is a first-order method with a constant step run for a fixed budget of 3000 iterations. Even without the penalty, the two would stop at different points along the same descent path.

The practical conclusion is that on this dataset the two implementations are equivalent as *classifiers* and differ only as *probability estimators*, and that the difference is entirely explained by regularisation. On a larger or noisier problem the library's shrinkage would be expected to pay off in generalisation; here, with the test set already saturated, it only shows up in the log-loss.

3 Question 2 — Support vector machines

3.1 Margins, support vectors and the role of C

For a linear SVM with decision function $f(x) = w^\top x + b$, the soft-margin primal problem is

$$\min_{w,b,\xi} \ \tfrac{1}{2}\|w\|^2 + C\sum_{i=1}^{m}\xi_i \qquad \text{s.t.} \qquad y_i\,(w^\top x_i + b) \ \geq \ 1 - \xi_i, \qquad \xi_i \geq 0. \tag{5}$$

The two canonical hyperplanes $f(x) = \pm 1$ are separated by the geometric margin $2/\|w\|$, so minimising $\|w\|^2$ maximises the margin. The slack variables ξ_i measure how far each point intrudes into or across the margin, and C is the price of that intrusion:

- **small** C — slack is cheap, so the optimiser buys a wide margin and tolerates many violations. High bias, low variance;

- **large** C — slack is expensive, so the margin is squeezed until almost every training point is classified correctly and outside it. Low bias, high variance.

Only the points with $\alpha_i > 0$ in the dual — those on the margin or violating it — enter the solution; these are the *support vectors*. Removing any other training point leaves the boundary unchanged. The number of support vectors is therefore both a summary of model complexity and a direct diagnostic of C.

3.2 (a) Geometric intuition on synthetic 2D data

Separable case. Sixty points are generated with `make_blobs` (two centres, `cluster_std`= 0.60, seed 42) and a linear SVM with $C = 1$ is fitted. The solution is $w = (0.1861, -0.1834)$, $b = 0.8813$, giving a margin width $2/\|w\| = 7.6540$, a training accuracy of 1.0000, and exactly 2 support vectors — one per class. The functional margin $|w^\top x + b|$ evaluated at both support vectors equals 1 to six decimal places, confirming that they lie precisely on the canonical hyperplanes.

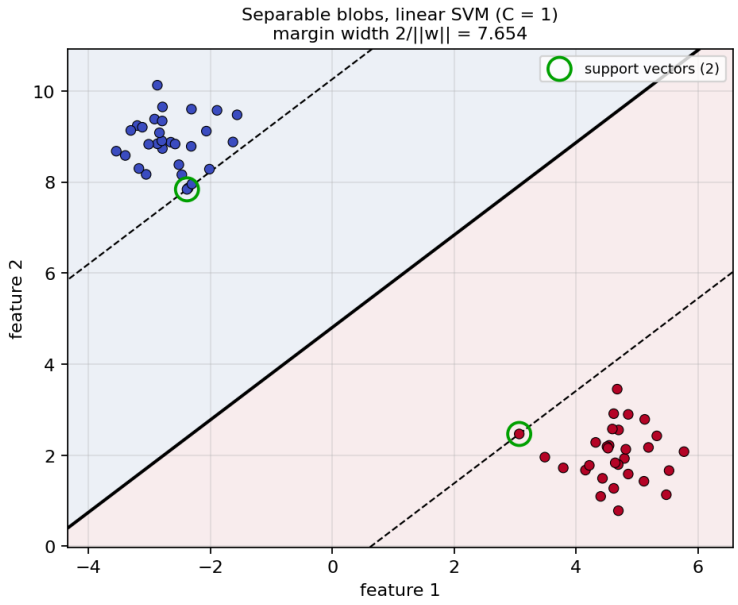


Figure 5: Linearly separable blobs. Solid line: decision boundary $f(x) = 0$. Dashed lines: the margin boundaries $f(x) = \pm 1$. Green rings: the 2 support vectors, one from each class.

Figure 5 illustrates the defining property of the SVM. The boundary is not placed anywhere between the clusters; it is placed to be as far as possible from both, and its position is fixed by only two of the sixty points. Every other point could be moved freely inside its own cluster without changing the solution at all — this is the sense in which the SVM is a *sparse*, geometry-driven classifier, in contrast to logistic regression, where every sample contributes a gradient term.

Overlapping case. Keeping the same seed and centres but raising `cluster_std` to 3.60 makes the two classes genuinely overlap, so no separating line exists. Three values of C spanning four orders of magnitude are compared in Table 7 and Figure 6.

Table 7: Effect of C on the overlapping blobs (60 samples, `cluster_std`= 3.60).

C	Margin $2/\|w\|$	$\|w\|$	# SVs	Train acc.	Misclassified
0.01	6.848	0.2921	22	0.9333	4
1.0	3.175	0.6298	12	0.9667	2
100.0	3.063	0.6530	11	0.9667	2

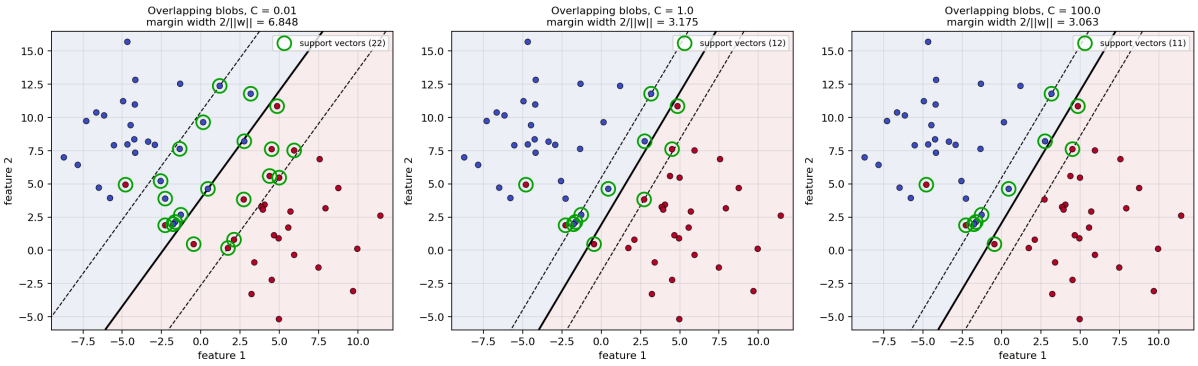


Figure 6: The same overlapping blobs fitted with $C = 0.01$, $C = 1$ and $C = 100$. As C grows the margin band narrows from 6.848 to 3.063 and the number of support vectors falls from 22 to 11.

The trend is exactly the one predicted by (5). At $C = 0.01$ the slack penalty is so weak that the optimiser opens a very wide corridor ($2/\|w\| = 6.848$) which swallows 22 of the 60 points, and it accepts 4 training misclassifications in exchange. At $C = 1$ the corridor halves to 3.175, only 12 points remain support vectors, and the training errors drop to 2. Raising C a hundredfold further to 100 changes almost nothing (3.063, 11 support vectors, still 2 errors): once C is large enough that the hinge term dominates, the solution is determined by the few genuinely overlapping points and saturates. The two remaining errors are points that lie deep inside the opposite cluster; no linear boundary can fix them, and a large C only distorts the boundary in a futile attempt to do so.

3.3 (b) Effect of regularisation on classification

A new two-dimensional dataset is generated with `make_classification` (250 samples, 2 informative features, one cluster per class, `class_sep`= 1.20, `flip_y`= 0.08, seed 42), so that the two clouds overlap and 8% of the labels are deliberately corrupted. It is split 80:20 with the same stratified splitter and the same seed, giving 200 training and 50 validation samples with balanced classes. Six values of C spanning five orders of magnitude are fitted; the results are in Table 8 and Figures 7–8.

Table 8: Linear SVM on the overlapping synthetic dataset, six values of C. The row selected on validation accuracy is in bold.

C	Margin $2/\|w\|$	$\|w\|$	# SVs	Train acc.	Val. acc.
0.001	9.294	0.2152	196	0.5200	0.5000
0.01	2.594	0.7711	118	0.9050	0.8800
0.1	1.453	1.3764	58	0.9500	0.9200
1.0	1.010	1.9804	40	0.9550	0.9400
10.0	0.904	2.2112	37	0.9400	**0.9600**
100.0	0.904	2.2113	37	0.9400	0.9600

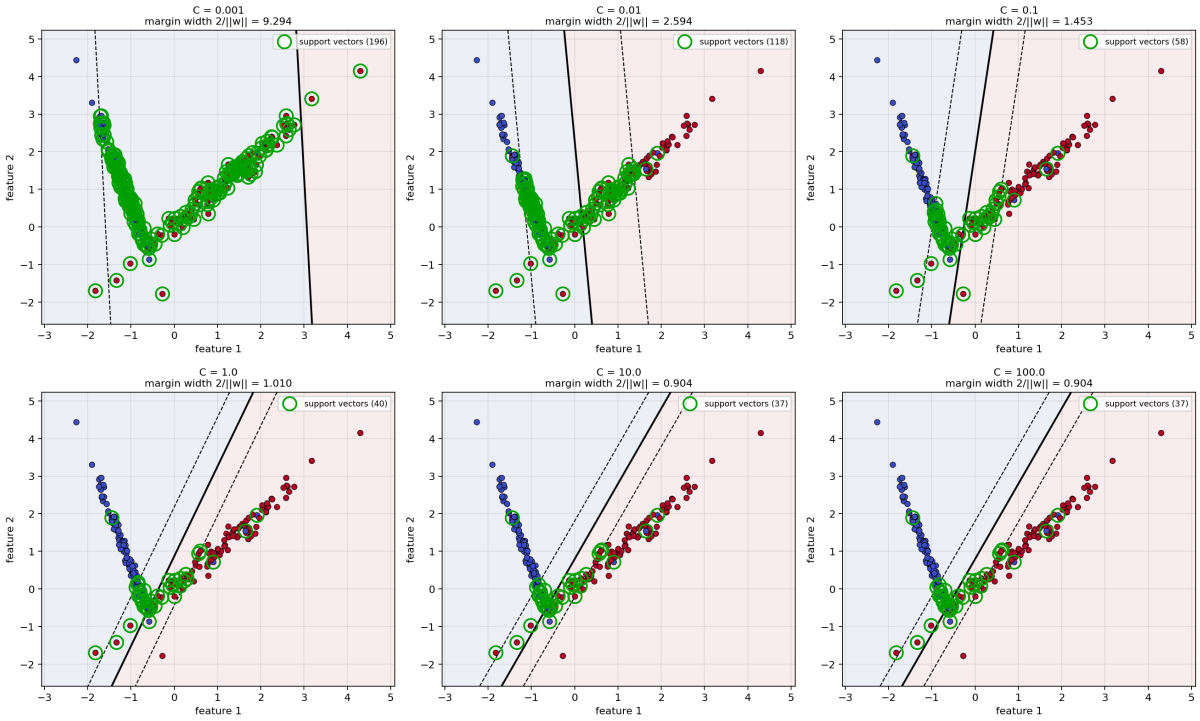


Figure 7: Decision boundary (solid), margin boundaries (dashed) and support vectors (green rings) for the six values of C. The margin band collapses and the boundary rotates into position as C increases.

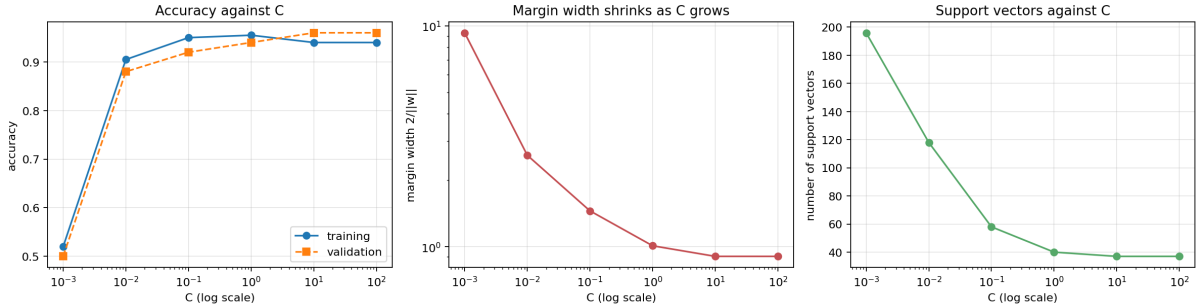


Figure 8: Summary of the sweep: accuracy, margin width and number of support vectors against C on logarithmic axes.

Three regimes are visible.

Severe underfitting ($C = 0.001$). The margin is 9.294 wide — larger than the entire spread of the data — so 196 of the 200 training points (98%) are support vectors. Minimising $\frac{1}{2}\|w\|^2$ completely dominates the tiny hinge term, w is driven towards zero, and the boundary is pushed to the edge of the cloud. Both accuracies collapse to chance level (0.520 training, 0.500 validation). This is the clearest possible demonstration that C too small is not "more robust" but simply broken.

Useful range ($C = 0.01$ to $C = 1$). Accuracy rises sharply and then levels off, the margin narrows monotonically from 2.594 to 1.010, and the support vector count drops from 118 to 40. Each increase of C lets the boundary attend to points nearer the overlap region instead of averaging over the whole cloud.

Saturation ($C = 10$ to $C = 100$). The margin width changes only in the fourth decimal place (0.90447 against 0.90443) and the support vector count is identical at 37. Once C is large enough that every remaining violation is an unavoidable one caused by the flipped labels,

further increases have nothing left to buy.

The best validation accuracy, 0.9600, is attained at $C = 10$ and $C = 100$; following the rule of breaking ties towards the smaller C (the wider margin and hence the simpler model), $C = 10.0$ is selected. Two details are worth noting. First, training accuracy is not monotone in C: it peaks at 0.955 for $C = 1$ and falls to 0.940 for $C \geq 10$. This is not a bug — the SVM minimises the hinge loss, not the 0/1 error, and a solution with a smaller hinge loss can have slightly more misclassifications. Second, the gap between training and validation accuracy is small throughout the useful range, which is what one expects from a two-parameter linear boundary on 200 points: with a model this rigid there is very little variance to control, and the whole effect of C is on bias.

3.4 (c) Full evaluation on the Breast Cancer Wisconsin dataset

Data. The dataset has 569 samples and 30 features, with 212 malignant (label 0) and 357 benign (label 1) cases, i.e. a 37.3%/62.7% balance. It is split 70:15:15 with the same from-scratch stratified splitter and seed 42 (Table 9), and standardised using training-set statistics only. Standardisation is not optional here: the raw features range from 0.05 (`mean fractal dimension`) to over 4000 (`worst area`), and an SVM penalises $\|w\|$ isotropically, so without scaling the large-magnitude features would be forced to carry tiny weights and would dominate the geometry.

Table 9: Stratified 70:15:15 split of the Breast Cancer Wisconsin dataset.

Subset	n	Malignant (0)	Benign (1)
Train	398	148	250
Validation	86	32	54
Test	85	32	53
Total	569	212	357

Selecting C. Six values of C from 10^{-3} to 10^{2} are fitted on the 398 training samples and scored on the 86 validation samples (Table 10, Figure 9).

Table 10: Model selection sweep on the validation set. The selected row is in bold.

C	# SVs	Margin $2/\|w\|$	Train acc.	Val. acc.	Val. ROC-AUC
0.001	195	6.3650	0.9447	0.9535	0.9988
0.01	90	2.9913	0.9698	0.9884	0.9988
0.1	46	1.5266	0.9849	0.9884	1.0000
1.0	31	0.6956	0.9925	**1.0000**	1.0000
10.0	27	0.2656	0.9950	0.9651	1.0000
100.0	22	0.0927	1.0000	0.9651	1.0000

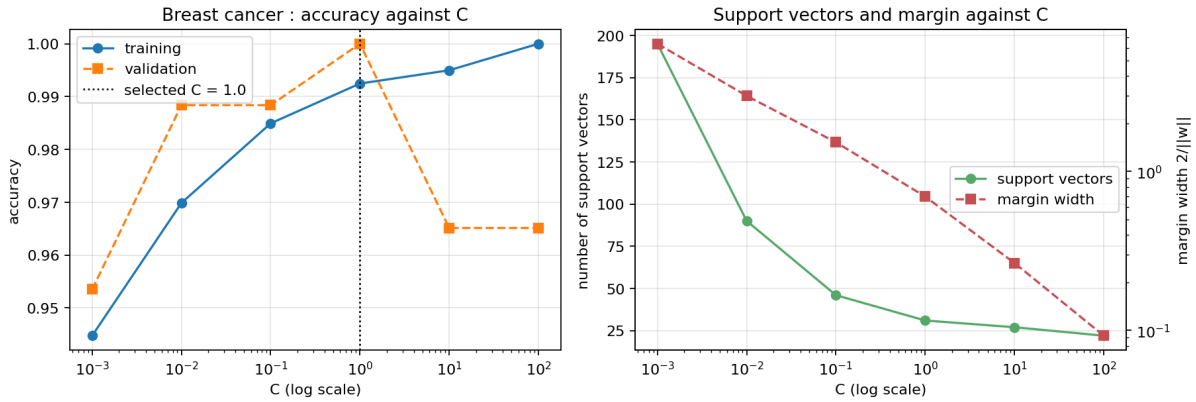


Figure 9: Left: training and validation accuracy against C, with the selected value marked. Right: the number of support vectors and the margin width, both falling monotonically as C increases.

Validation accuracy rises from 0.9535 at $C = 0.001$ to a maximum of 1.0000 at $C = 1$ and then falls back to 0.9651 for $C \geq 10$, so the selected value is $\boxed{C = 1.0}$. The selected model has 31 support vectors (14 malignant and 17 benign), which is only 7.8% of the 398 training points, and a margin width of 0.6956. That sparsity is the headline geometric fact: the entire 30-dimensional decision boundary is pinned down by 31 patients, all of them lying in the narrow band where malignant and benign cases are hard to tell apart.

Across the sweep the number of support vectors falls from 195 (nearly half the training set) at $C = 0.001$ to 22 at $C = 100$, while the margin shrinks from 6.3650 to 0.0927, a factor of almost 70. At $C = 100$ the training accuracy reaches 1.0000 — the model interpolates the training data — yet validation accuracy is *lower* than at $C = 1$. That divergence between training and validation performance is textbook overfitting, and it is exactly what validation-based selection is for.

Test-set performance of the selected model. The selected linear SVM is evaluated once on the 85 untouched test samples. Taking benign as the positive class (the class scored positively by the decision function):

Accuracy	0.9529
Precision	0.9298
Recall	1.0000
F1-score	0.9636
ROC-AUC (from decision-function scores)	0.9982

Table 11: Per-class test metrics of the selected linear SVM ($C = 1.0$).

Class	Support	Precision	Recall	F1
Malignant (0)	32	1.0000	0.8750	0.9333
Benign (1)	53	0.9298	1.0000	0.9636
Macro average	85	0.9649	0.9375	0.9485

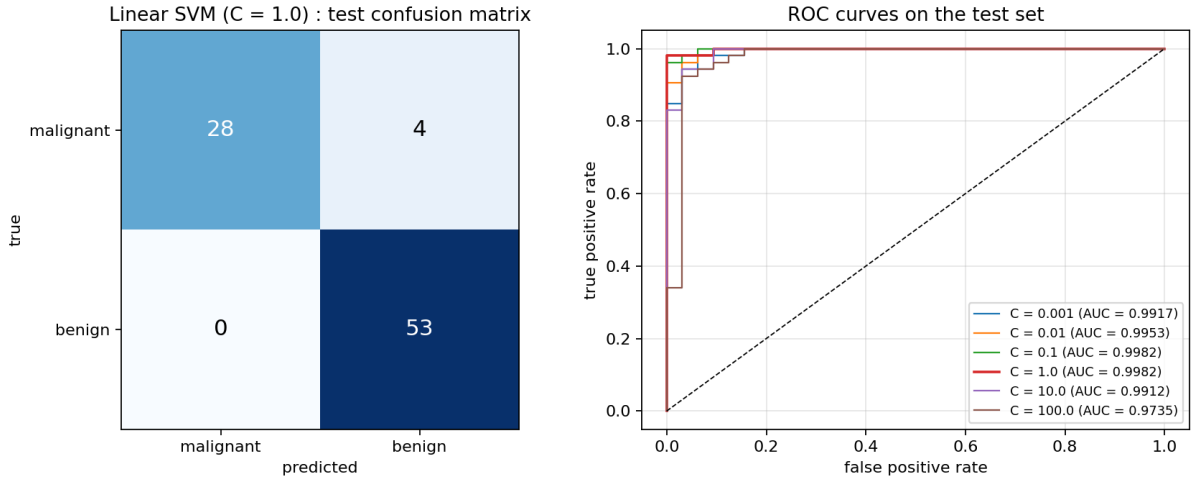


Figure 10: Left: test confusion matrix of the selected model. Right: ROC curves on the test set for all six values of C, computed from the raw decision-function scores.

The confusion matrix is $\left[\begin{smallmatrix} 28 & 4 \\ 0 & 53 \end{smallmatrix}\right]$: all 53 benign cases are identified, and 4 of the 32 malignant cases are misclassified as benign. This asymmetry deserves comment. The reported recall of 1.0000 refers to the benign class; for the malignant class the recall is only 0.8750 while its precision is 1.0000. In a diagnostic setting those 4 errors are false negatives — the expensive kind — so a deployed version of this classifier would not use the default threshold at $f(x) = 0$. The ROC-AUC of 0.9982 shows that the *ranking* produced by the decision function is nearly perfect, which means a threshold shifted towards the malignant side could recover most of those cases at the cost of a few benign false alarms. This is precisely why the ROC-AUC is computed from the continuous scores rather than from the hard labels: it measures the quality of the score independently of where the threshold happens to sit.

Comparison across C on the test set. Table 12 repeats the evaluation for every C considered.

Table 12: Test-set metrics for every value of C considered (positive class: benign). The row selected on the validation set is in bold.

C	# SVs	Accuracy	Precision	Recall	F1	ROC-AUC
0.001	195	0.8824	0.8413	1.0000	0.9138	0.9917
0.01	90	0.9647	0.9464	1.0000	0.9725	0.9953
0.1	46	0.9529	0.9298	1.0000	0.9636	0.9982
1.0	31	0.9529	0.9298	1.0000	0.9636	0.9982
10.0	27	0.9412	0.9138	1.0000	0.9550	0.9912
100.0	22	0.9412	0.9138	1.0000	0.9550	0.9735

Test accuracy is worst at $C = 0.001$ (0.8824) — the over-wide margin underfits badly — rises to 0.9647 at $C = 0.01$, and then settles around 0.94–0.95. ROC-AUC peaks at 0.9982 for $C \in \{0.1, 1\}$ and degrades at both ends, to 0.9917 at $C = 0.001$ and 0.9735 at $C = 100$, so the intermediate values give the best-calibrated ranking as well as the best accuracy.

One honest observation: the value chosen on the validation set ($C = 1$, 0.9529 test accuracy) is *not* the best-performing value on the test set ($C = 0.01$, 0.9647). With only 86 validation samples, a validation accuracy of 1.0000 is a noisy estimate — the difference between $C = 0.01$, $C = 0.1$ and $C = 1$ there is one or two samples — and picking the argmax of a noisy criterion

generally overstates how good the winner is. The gap between 0.9647 and 0.9529 is itself one test sample out of 85, so it should not be over-interpreted either. The methodologically correct response is not to re-select C on the test set, which would destroy the meaning of the held-out estimate, but to note that a k-fold cross-validation over the combined training and validation data would give a far more stable selection than a single 86-sample split. Within the flat region $C \in [0.01, 1]$ the choice barely matters; what matters is staying out of the two extremes.

4 Conclusions

1. **The from-scratch softmax model is correct.** Its analytic gradient matches central finite differences to a relative error of 5.53×10^{-10}, and with $K = 2$ and the reference column fixed at zero it reproduces binary logistic regression to 2.22×10^{-16} — the softmax formulation genuinely contains the sigmoid as a special case rather than merely resembling it.

2. **The learning rate acts as an implicit regulariser.** All four rates converged and all four passed through nearly the same minimum validation cross-entropy, but the larger two then climbed back away from it by 0.018 and 0.035 within the same iteration budget (Figure 2). On separable, unregularised data the training cost can always be pushed lower by growing the weights; validation cross-entropy, not training cost, is what detects this.

3. **Our implementation and the library agree as classifiers and differ as probability estimators.** Identical predictions on all 26 test samples and identical accuracy, precision, recall and F1 (1.0000), but log-loss 0.007035 against 0.022759 and weight norm 4.875 against 3.338. The whole difference is the L2 penalty that the library applies by default.

4. **The SVM solution is sparse and geometric.** Two points out of sixty fix the boundary in the separable synthetic case; 31 patients out of 398 fix it on the breast cancer data. Everything else in the training set is irrelevant to the fitted model.

5. C **trades margin width against training error, and both extremes fail.** Across every experiment, increasing C narrowed the margin and reduced the number of support vectors monotonically. Too small and the model underfits catastrophically (chance-level accuracy at $C = 0.001$ in Q2(b), 0.8824 test accuracy in Q2(c)); too large and it interpolates the training data while validation accuracy falls. The useful values sat in a broad flat middle region.

6. **Threshold-free metrics matter.** The selected SVM reaches 0.9529 accuracy and 0.9982 ROC-AUC, but misses 4 of 32 malignant cases at the default threshold. The near-perfect AUC shows the failure lies in the threshold, not in the score — which is why a clinical deployment would tune the operating point rather than the model.